

ADVANCE PROBLEM PROGRAMMING

SLO 2: Class Design & Inheritance

1. Design a Class Hierarchy: Vehicle → Car → ElectricCar

```
class Vehicle {  
    void start() {  
        System.out.println("Vehicle starts");  
    }  
}  
  
class Car extends Vehicle {  
    void drive() {  
        System.out.println("Car is driving");  
    }  
}  
  
class ElectricCar extends Car {  
    void charge() {  
        System.out.println("Electric car is charging");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        ElectricCar e = new ElectricCar();  
  
        e.start();  
    }  
}
```

```
        e.drive();
        e.charge();
    }
}
```

Output:

Vehicle starts

Car is driving

Electric car is charging

Here, Car inherits from Vehicle, and ElectricCar inherits from Car. This forms a **multilevel inheritance hierarchy**.

2. Create a Superclass Employee and Subclass Manager

```
class Employee {
    String name;
    double salary;

    Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    void display() {
        System.out.println("Name: " + name);
        System.out.println("Salary: " + salary);
    }
}
```

```
class Manager extends Employee {  
    String department;  
  
    Manager(String name, double salary, String department) {  
        super(name, salary);  
        this.department = department;  
    }  
  
    void displayManager() {  
        display();  
        System.out.println("Department: " + department);  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Manager m = new Manager("Ravi", 50000, "IT");  
  
        m.displayManager();  
    }  
}
```

Output:

Name: Ravi

Salary: 50000.0

Department: IT

Here, Employee is the superclass and Manager is the subclass. The Manager class inherits the properties and methods of Employee.

3. Override a Method in Java and Explain the Use of super

Method overriding occurs when a subclass provides its own implementation of a method already defined in the superclass.

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes a sound");  
    }  
}
```

```
class Dog extends Animal {  
    @Override  
    void sound() {  
        super.sound();  
        System.out.println("Dog barks");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.sound();  
    }  
}
```

Output:

Animal makes a sound

Dog barks

The super keyword is used to refer to the **immediate parent class**.

In this example:

```
super.sound();
```

calls the sound() method of the Animal class from inside the overridden method of Dog.

Uses of super:

1. To call a superclass method.
2. To access superclass variables.
3. To call a superclass constructor.